Fusion360ParametricExportScript

Public

About

Simple Python script to export multiple models based on json configuration.

Readme
MIT license
Activity
1 star
0 watching
0 forks
Report repository

Releases

No releases published

Packages

No packages published

Contributors 1

BrezinaAdam Adam Brezina

Languages

Python 100.0%

m... 1 Branch 0 Tags Go to file T Go to file Add file Code

BrezinaAdam added multiformat export with examples fbbacdd · 10 months ago

sample-config	added multiformat export with examples	10 months ago
.gitignore	better readme	10 months ago
LICENSE	Initial commit	10 months ago
ParametricExport.manifest	Initial commit of ParametricExport script	10 months ago
ParametricExport.py	added multiformat export with examples	10 months ago
	added multiformat export with examples	10 months ago

README.md

icon.svg	Initial commit of ParametricExport script	10 months ago

README | MIT license

Parametric Export for Fusion 360

This Fusion 360 script, `ParametricExport.py`, allows you to automate the process of exporting multiple variations of your parametric designs. Instead of manually changing parameters and exporting STL files one by one, you can define all desired variations in a simple JSON configuration file and let the script do the work for you.

Features

- **Configuration-Driven:** Define all your export variations in a simple, human-readable JSON file. No need to edit the Python script for new projects.
- **Batch Processing:** Iterates through every possible combination of the parameters you specify.
- **Dynamic File Naming:** Automatically generates descriptive filenames based on the parameters used for each variant.
- **Folder Grouping:** Automatically organizes exported files into subdirectories based on the parameters of your choosing.
- **User-Friendly:** A progress bar keeps you informed of the export status and allows you to cancel the operation at any time.

How to Use

1. **Open Your Design:** Open the Fusion 360 design you wish to export.
2. **Run the Script:** In Fusion 360, go to the `UTILITIES` tab and select `Scripts and Add-Ins`. Click the `Add` button (the green `+`), navigate to the `ParametricExport.py` file, and click `Run`.
3. **Select Configuration:** A file dialog will appear. Select the JSON configuration file that defines the exports you want to perform (e.g., `handle_config.json`).
4. **Export:** The script will begin processing all the variants defined in your file. A progress bar will show the status.
5. **Done:** Once complete, all your exported STL files will be in the `output` directory (or the directory specified in your config), neatly organized.

The JSON Configuration File

The power of this script lies in the JSON configuration file. Here is a breakdown of its structure.

Top-Level Keys

- `outputDirectory`: (String) The name of the folder where the exported files will be saved.
- `bodiesToExport`: (List of Strings) A list of the exact names of the bodies you want to export.
- `exportOptions`: (Object, Optional) A powerful section to control the export format and naming.
- `parametersToIterate`: (Object) An object where you define the User Parameters from your Fusion 360 design that you want to change.

`exportOptions` Object (New in v2)

This optional object gives you fine-grained control over the output.

- `fileType`: (String | String[], Optional) The type of file to export. Can be a single string or an array of strings (e.g., `["STL", "STEP"]`). Supported values are `"STL"`, `"STEP"`, and `"IGES"`. Defaults to `"STL"`.

- `stlQuality` : (String, Optional) The mesh quality for STL exports. Supported values are `"High"` , `"Medium"` , and `"Low"` . Defaults to `"Medium"` .
- `fileNameTemplate` : (String, Optional) A template for the output filename. You can use placeholders that correspond to the parameter names in your design.
  - `{bodyName}` : The name of the body being exported.
  - `{YourParameterName}` : The value of a specific parameter for that variant (e.g., `{Size}` , `{Thickness}`).
  - `{params}` : A default, auto-generated string of all parameter values (e.g., `Size_2-Thickness_5`).
- `forceRecompute` : (Boolean, Optional) If `true` , the script will force Fusion 360 to recompute the entire model after changing parameters for each variant. This ensures complex changes are captured but can be disabled for faster exports on simple models. Defaults to `true` .

`parametersToIterate` Object

Each key in this object must match the **exact name** of a User Parameter in your Fusion 360 design (e.g., `"Thickness"`). The value for each key is another object with the following properties:

- `variants` : (List of Numbers/Strings) A list of all the values you want to assign to this parameter. The script will iterate through each of these.
- `grouping` : (Boolean, Optional) If you set this to `true` , this parameter will be used to create a subfolder in the output directory. Defaults to `false` .

Sample Configurations

The `sample-config` directory provides examples for different use cases.

`gridfinityHandles.json`

This file is a good example of a complex, multi-parameter export.

- **Exports:** Both `STL` and `STEP` files for each variant of a parametric handle.
- **Grouping:** Creates a combined folder for each `Size` and `HandleWidth` pair (e.g., `export/Size-2-HandleWidth-16`).
- **Filename:** Generates a detailed filename from multiple parameters, like: `Handle-s2-a105-th5mm-w16mm.stl` .

```
{
  "outputDirectory": "export",
  "exportOptions": {
    "fileType": ["STL", "STEP"],
    "stlQuality": "High",
    "fileNameTemplate": "{bodyName}_size-{Size}_angle-{HandleAngle}_thick-{Thickness}mm_width-{HandleWidth}mm",
    "forceRecompute": true
  },
  "bodiesToExport": [
    "Handle",
    "Handle-wScrewHole"
  ],
  "parametersToIterate": {
    "Size": {
      "variants": [2, 3, 4, 5],
      "grouping": true
    },
    "HandleAngle": {
      "variants": [105, 115, 125, 135]
    },
    "HandleWidth": {
      "variants": [16, 23],
      "grouping": true
    },
    "Thickness": {
```

```
      "variants": [5, 7, 9]
    }
  }
}
```

`gridfinityUnderDeskDrawers.json`

This file demonstrates a simpler, single-parameter export.

- **Exports:** `STEP` files for a parametric drawer.
- **Grouping:** Creates a folder for each `COLUMNS` value (e.g., `drawer-exports/COLUMNS-2`).
- **Filename:** Generates a name based on the body and the parameter, like `Drawer-c2.step`.
- **Example Path:** `drawer-exports/COLUMNS-2/Drawer-Button-c2.step`

```
{
  "outputDirectory": "drawer-exports",
  "exportOptions": {
    "fileType": "STEP",
    "fileNameTemplate": "{bodyName} with {COLUMNS}x4 baseplate",
    "forceRecompute": true
  },
  "bodiesToExport": [
    "Drawer",
    "Drawer-Button",
    "Drawer-Handle"
  ],
  "parametersToIterate": {
    "COLUMNS": {
      "variants": [1, 2, 3, 4, 5],
      "grouping": true
    }
  }
}
```